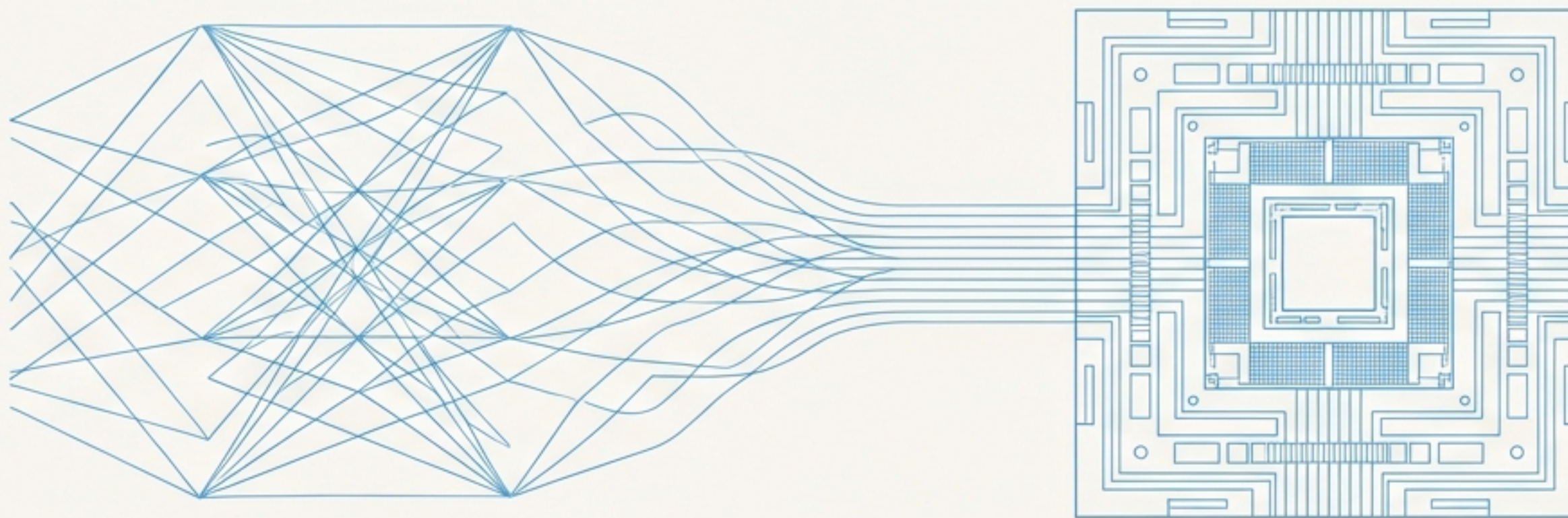
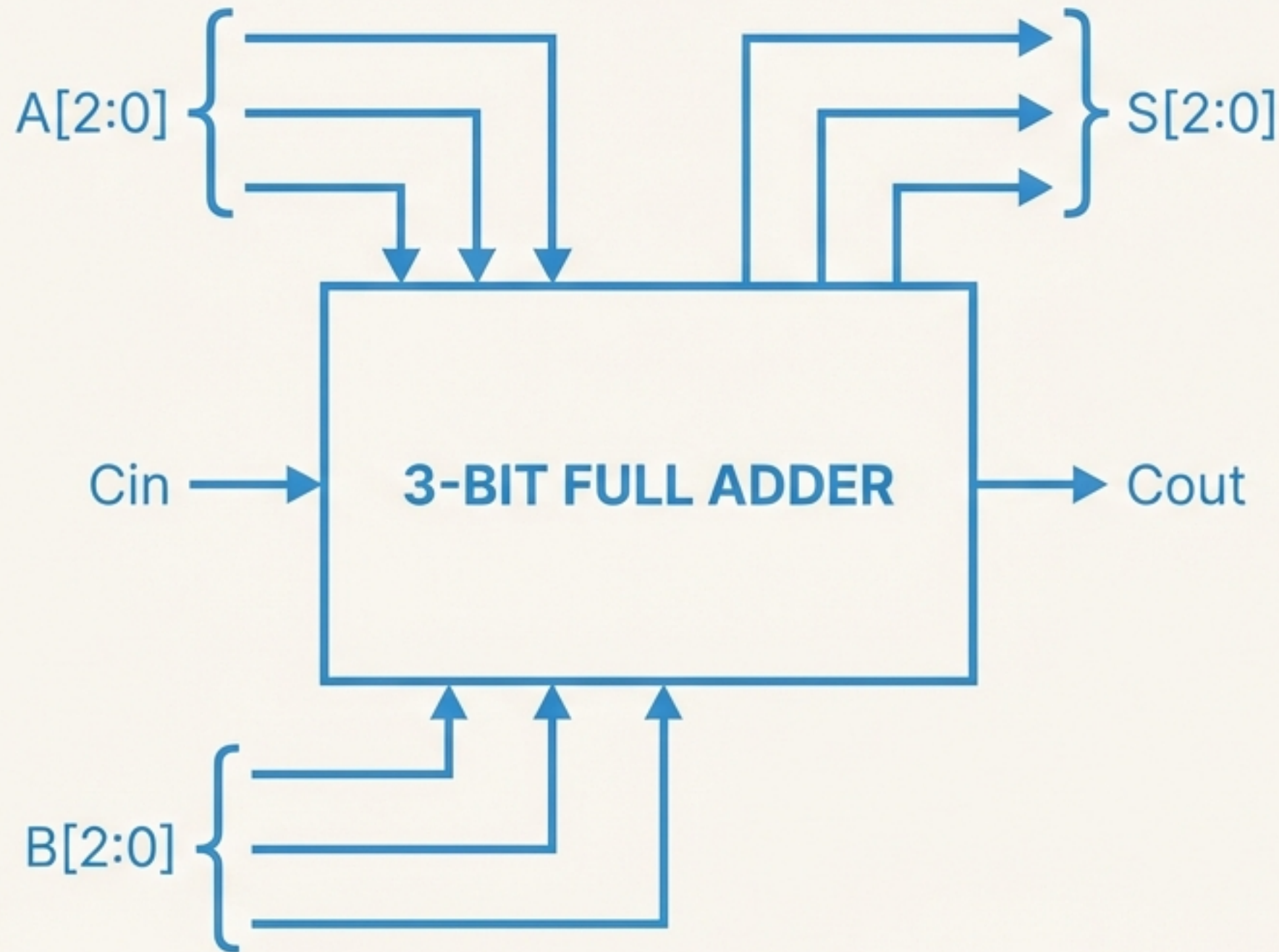


The Next Frontier: AI in Chip Design



- Large Language Models are transforming software development, automating code generation and complex problem-solving.
- The semiconductor industry presents a new frontier, with the potential to automate aspects of Electronic Design Automation (EDA).
- This presentation explores a case study in AI-driven hardware generation, revealing a critical distinction between apparent success and true correctness.

Our Case Study: The Fundamental 3-Bit Adder



To test the capabilities of an LLM, we use a foundational digital logic circuit: a 3-bit binary adder.

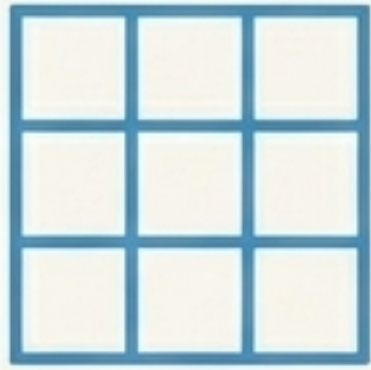
Function:

It takes two 3-bit numbers (A , B) and a carry-in (C_{in}), producing a 3-bit sum (S) and a carry-out (C_{out}).

Significance:

While simple, its behaviour is precisely defined and non-trivial, making it an ideal benchmark for logical correctness. The complete truth table contains 128 possible input combinations.

From Function to Form: Multiple Representations of Hardware



Truth Table

Defines functional behaviour. (e.g.,
``adder_3-bit_tab.csv``)



Verilog HDL

Describes the circuit for simulation and synthesis. (e.g.,
``adder_3-bit.v``)



Conjunctive Normal Form (CNF)

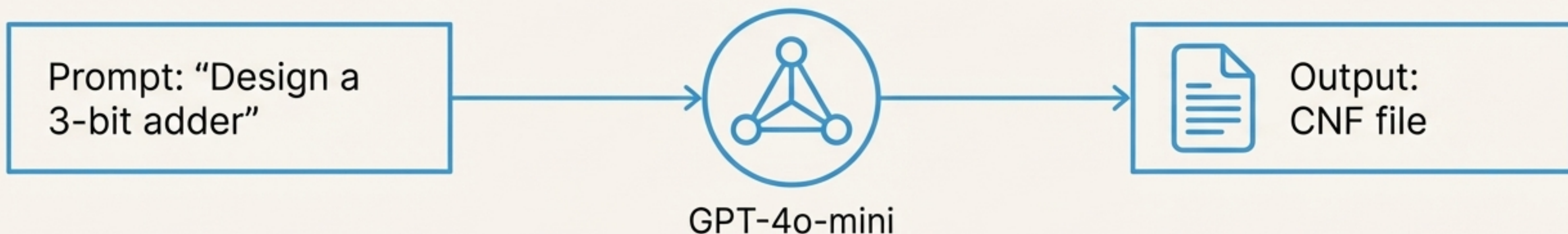
Enables formal verification and logical proof using SAT solvers. (e.g.,
``adder_3-bit.cnf``)



Benchmark Format

A standard format for synthesis tools and academic benchmarks. (e.g.,
``adder_3-bit.bench``)

The Experiment: Can an LLM Generate Verifiably Correct Hardware Logic?



Objective: To use a Large Language Model to generate the Conjunctive Normal Form (CNF) representation of a hardware design.

Model: GPT-4o-mini.

Process: The model was prompted to create CNF equations for a 3-bit adder. The output's correctness was then evaluated.

Why CNF?: Generating correct CNF is a rigorous test of an AI's ability to capture precise, formal boolean relationships, not just descriptive code.

The First Hurdle: Syntactic Failures

```
1 c variable -1 invalid_name
2 p cnf var /2 clauses
3 (A0 \\ B0) & (Cout_1)
```

- Initial outputs from the model were syntactically invalid.
- The generated CNF expressions contained illegal characters, including backslashes (`\`), dashes (`-`), and underscores (`\_`) within variable names.
- These syntax errors caused parsing scripts to fail immediately. The output was unusable for any downstream verification tools.

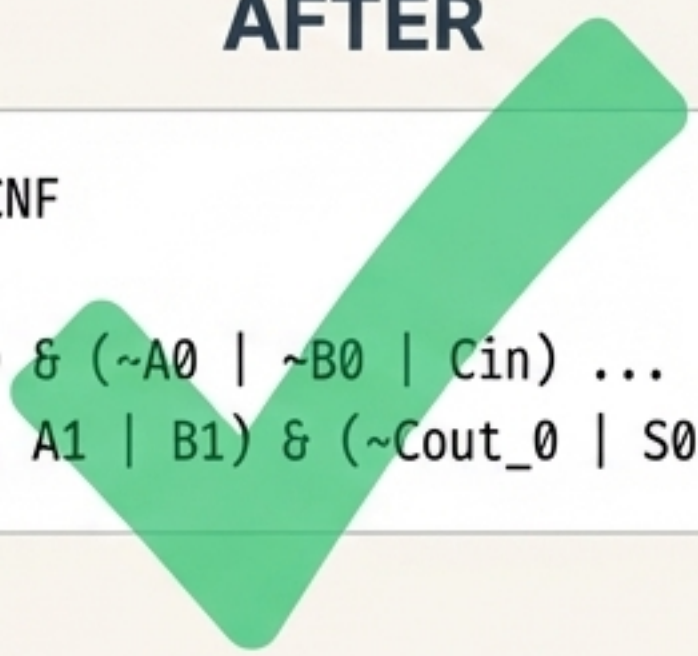
Apparent Success Through Prompt Refinement

BEFORE

```
1 c variable -1 invalid_name
2 p cnf var /2 clauses
3 (A0 \\ B0) & (Cout_1)
```

AFTER

```
c 3-bit Adder CNF
p cnf 6 12
(A0 | B0 | Cin) & (~A0 | ~B0 | Cin) ...
(Cout_0 | ~S0 | A1 | B1) & (~Cout_0 | S0 | A1 | B1) ...
```

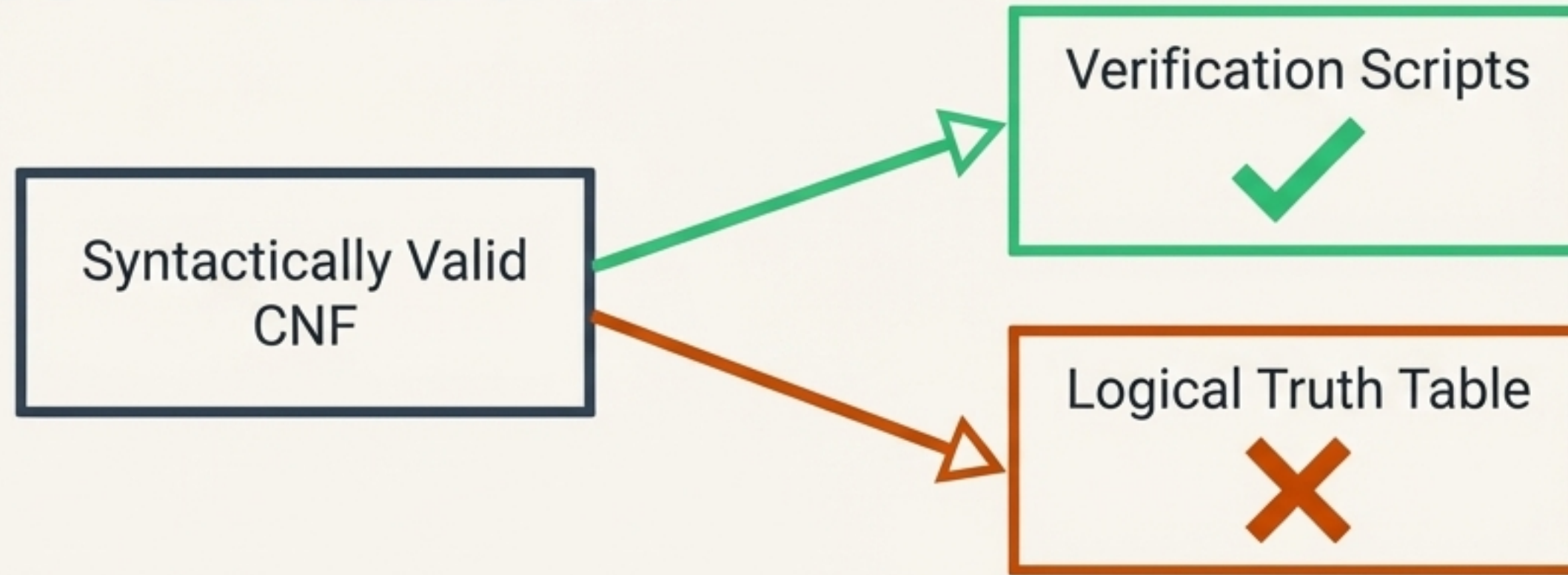


Through an iterative process, the prompts were refined to enforce strict rules:

- Restrict special characters in variable names.
- Enforce consistent naming conventions.

Result: The model successfully produced clean, syntactically valid CNF files. These files passed all preliminary parsing and I/O script tests, confirming they were structurally correct and processable by automated tools.

Beyond Syntax: The Critical Need for Semantic Validation



A syntactically valid file is not necessarily a logically correct one. The structure can be perfect, but the underlying boolean relationships can be wrong.

The only way to confirm correctness is through semantic validation: comparing the complete input/output behaviour of the generated logic against the ground truth.

This was achieved by generating a full truth table from the LLM's CNF and comparing it to the expected truth table of a 3-bit adder.

The Adder's Flaw: A Fundamental Failure of Logic

Expected Behaviour

Input: $A=000$, $B=000$, $C_{in}=0$

Calculation: $0 + 0 + 0$

Expected Output:
Sum= 000 , $C_{out}=0$

LLM-Generated Behaviour

Input: $A=000$, $B=000$, $C_{in}=0$

Calculation: $0 + 0 + 0$

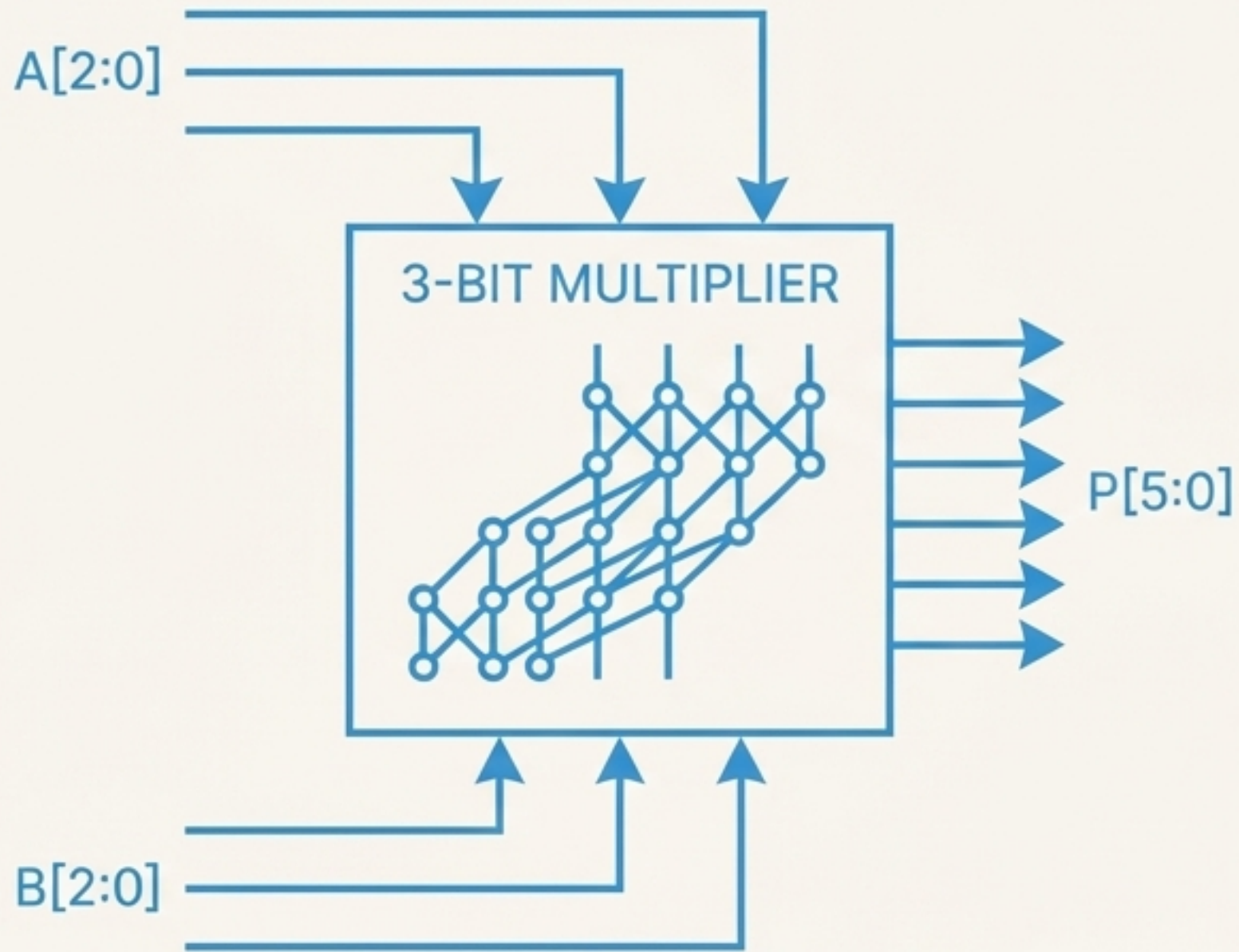
Generated CNF Output:
Sum= 010 , $C_{out}=1$

The LLM's CNF erroneously asserts a carry-out and calculates the sum as '2' when both inputs are zero. The most basic case fails.

Visualising the Mismatch

	A2	A1	A0	B2	B1	B0	Cin		S2	S1	S0	Cout
Ground Truth	0	0	0	0	0	0	0		0	0	0	0
LLM Generated	0	0	0	0	0	0	0		0	1	0	1

Escalating the Test: The 3-Bit Multiplier



To determine if the logical errors were specific to the adder, the experiment was repeated with a 3-bit multiplier.

The same process was followed: iterative prompt refinement yielded a syntactically valid CNF file.

This syntactically correct file was then subjected to semantic validation against the multiplier's expected truth table.

A Total Inversion of Logical Behaviour

Expected Behaviour

Input: A=000, B=000

Calculation: 0×0

Expected Output:

Product=000000

LLM-Generated Behaviour

Input: A=000, B=000

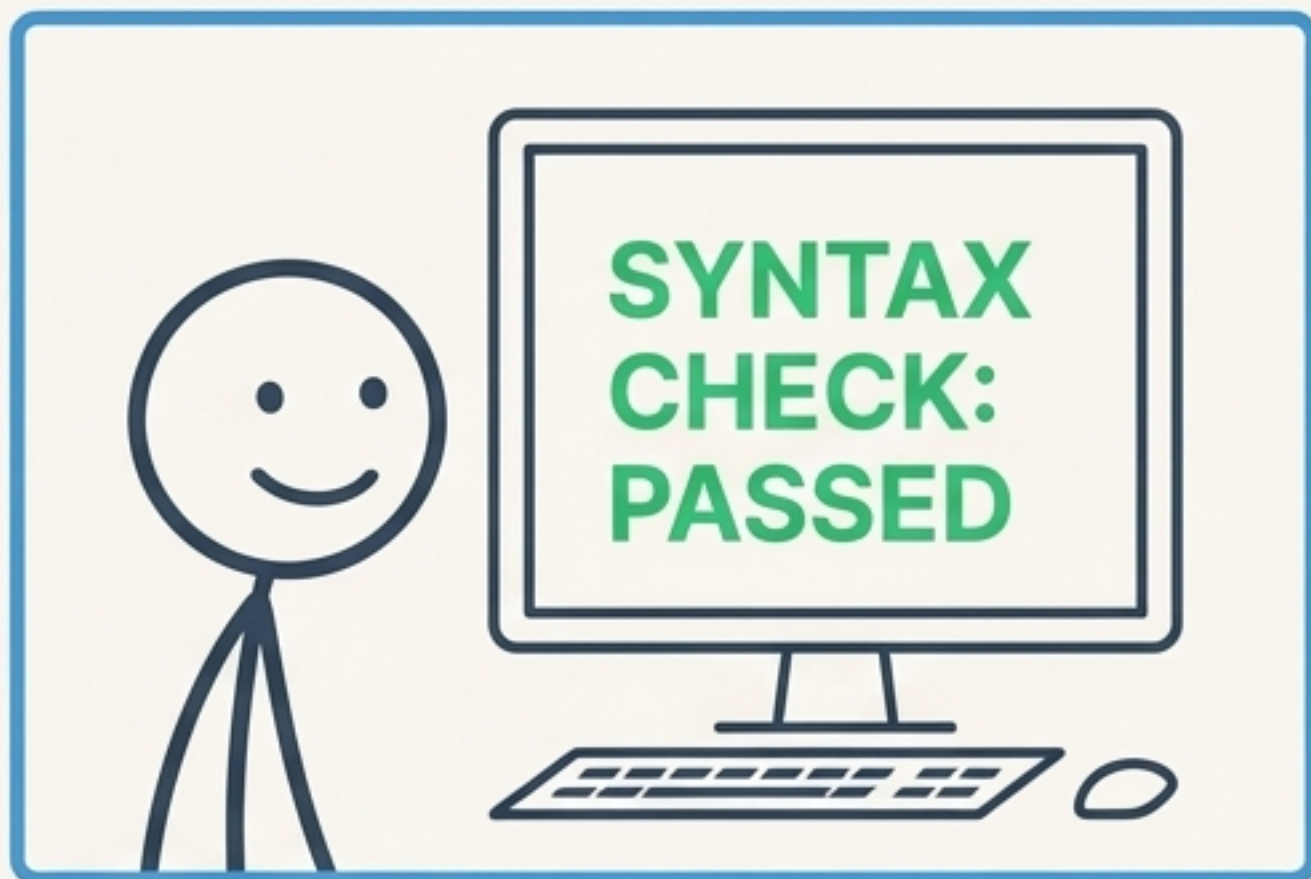
Calculation: 0×0

Generated CNF Output:

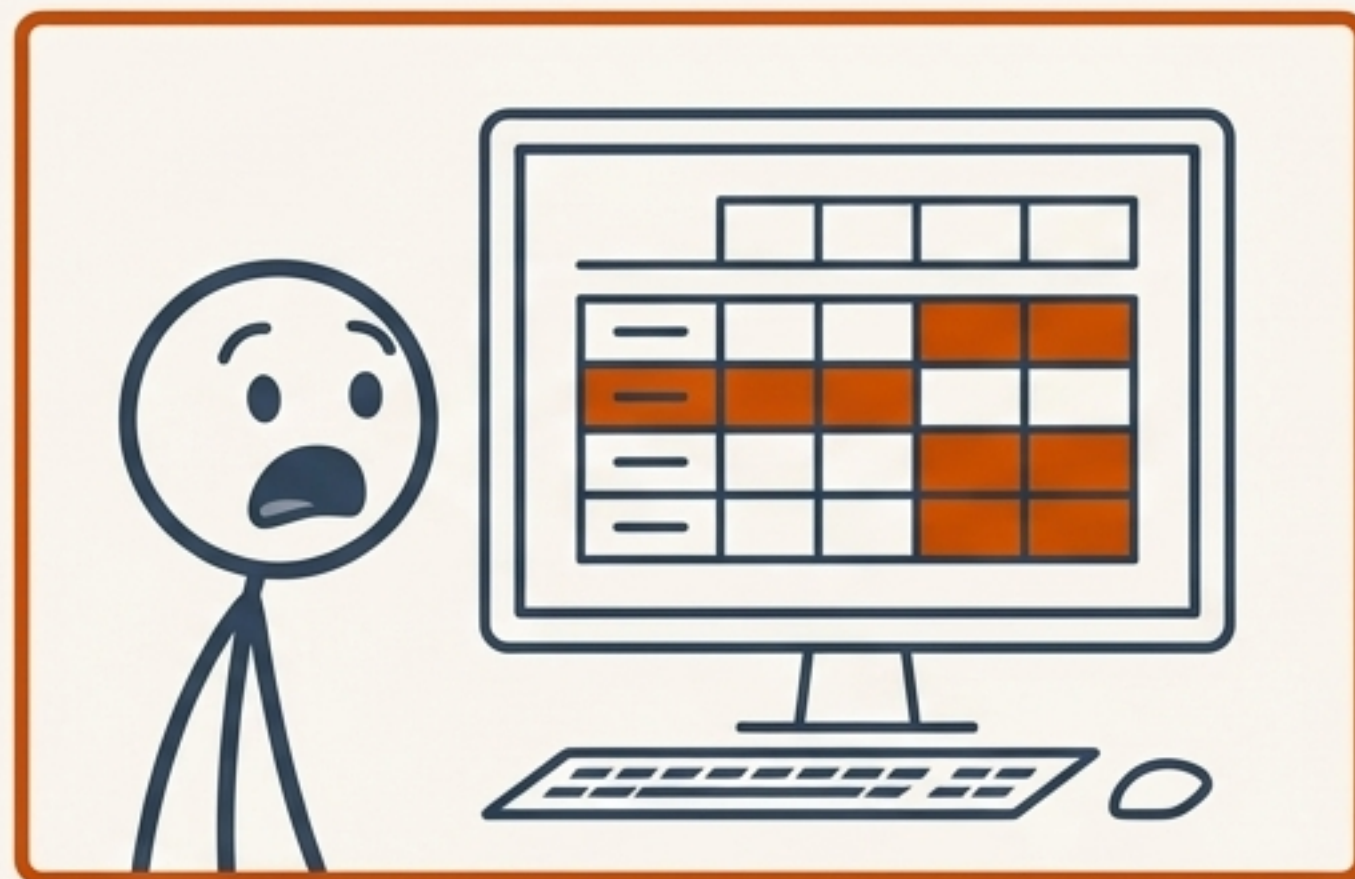
Product= **111111**

For zero operands, the generated CNF drives all output bits high. This is not just an error; it is a “**severe semantic misrepresentation**” and a **complete inversion of the circuit’s fundamental logic**.

The Core Lesson: Syntactic Success Does Not Imply Logical Fidelity



Passing basic script checks
can be misleading.

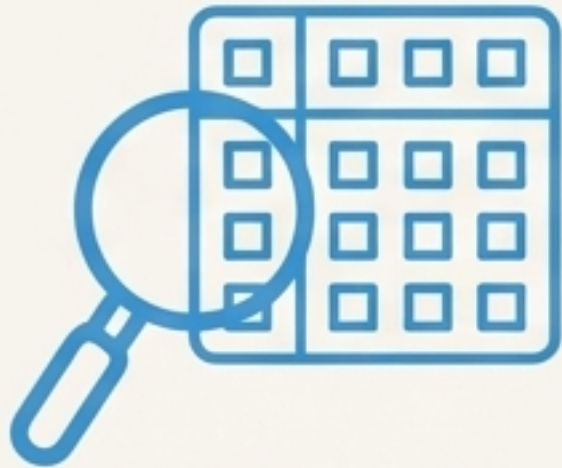


Complete validation reveals deeper
logical inconsistencies.

- Prompt refinement can successfully fix syntax but offers no guarantee of semantic correctness.
- LLMs can generate plausible outputs that satisfy superficial checks but fail to represent the intended functionality faithfully.

The Imperative for Rigorous Verification

To ensure logical soundness, AI-generated hardware designs must be complemented by robust verification strategies.



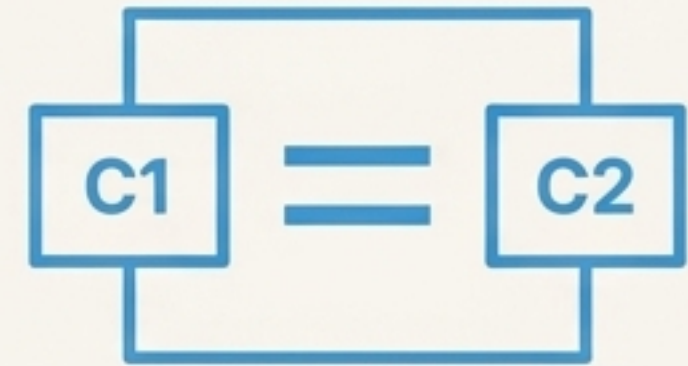
Exhaustive Simulation

Validating the full truth table for smaller designs.

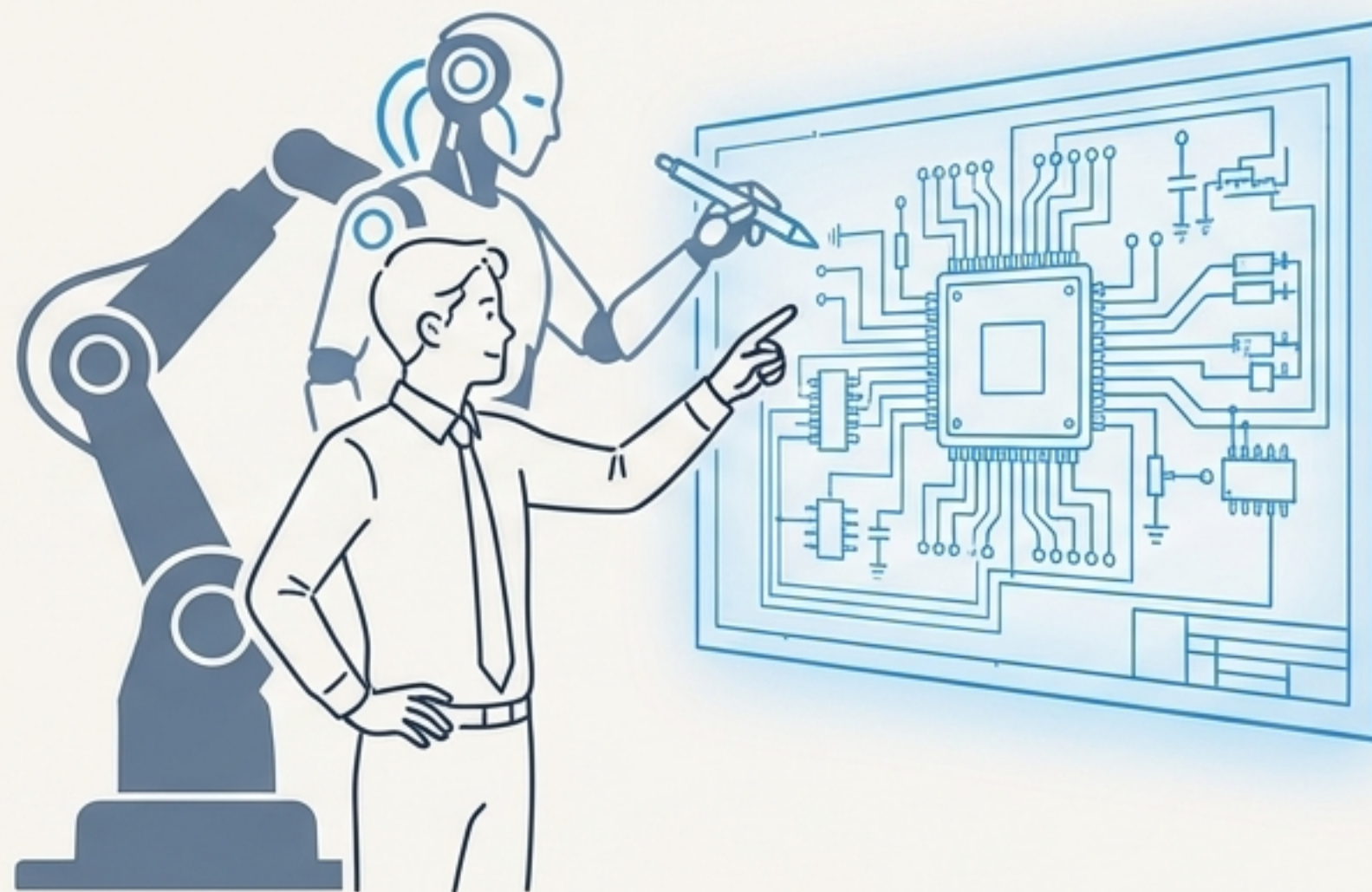
$$\forall (x \rightarrow y)$$

Formal Verification / Equivalence Checking

Mathematically proving that the generated logic is equivalent to a trusted 'golden' reference model.



Augmentation, Not Automation: The Role of AI in High-Stakes Engineering



- Large Language Models are powerful tools for augmenting the engineering process, but they do not yet replace the need for domain expertise and rigorous discipline.
- The critical challenge is not just in generation, but in building automated systems that can perform deep semantic validation.
- For safety-critical domains like chip design, a human-in-the-loop approach, backed by formal verification, remains essential.